Trabajo Integrador — Parte 1

Sistema de Monitoreo de Sensores (SCADA simplificado)

Laboratorio III — Universidad de Palermo
Alumna: Alina Picc
Alcance: Parte 1 — Adquisición y procesamiento concurrente (Módulos 1 a 8)
Modalidad: Individual

1. Introducción y alcance de esta entrega

Este documento corresponde a la **Parte 1** del Trabajo Integrador de Laboratorio III. Segun la consigna, el sistema completo simula un SCADA (Supervisión, Control y Adquisición de Datos) simplificado: recolecta mediciones de varios sensores, las procesa en paralelo, mantiene estadísticas agregadas accesibles en tiempo real y (en la Parte 2) reacciona a señales del operador.

El enunciado divide el trabajo en dos partes. Esta entrega cubre exclusivamente la **Parte 1: el camino de datos hasta el procesamiento concurrente**: los sensores producen mediciones, viajan por una cola de mensajes, un *pool* de worker threads las procesa y actualiza estadísticas en memoria compartida, y un segundo proceso puede consultarlas. Las alertas con *logger* y el manejo de señales (SIGINT/SIGALRM/SIGUSR1) quedan explícitamente para la Parte 2, por lo que no se implementaron aquí.

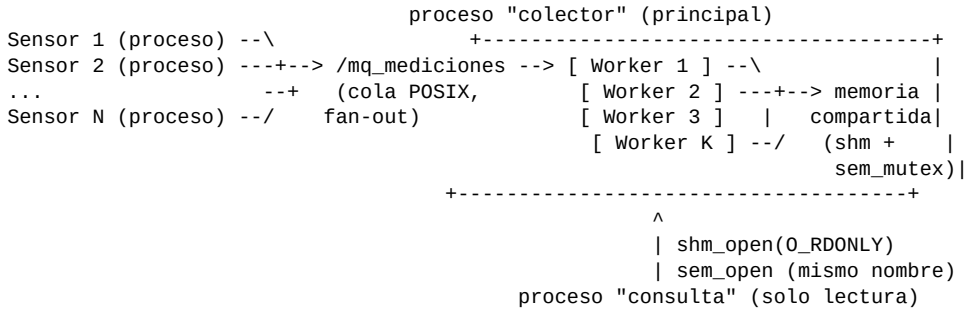
1.1 Requisitos puntuales del enunciado (Parte 1)

La consigna pide, textualmente, siete puntos. Se resumen aquí y se detalla su cumplimiento en la sección 5:

- Sensores: el colector crea una cantidad configurable de procesos sensores.
- Envío por cola de mensajes: las mediciones viajan de los sensores al colector por una cola de mensajes.
- Procesamiento concurrente: *threads workers* consumen la cola (fan-out) y procesan; los datos compartidos se protegen con mutex y/o semáforo.
- Estadísticas en memoria compartida: cantidad, suma/promedio, mínimo, máximo y último valor, por sensor y globales.
- Configuración por línea de comandos: al menos cantidad de sensores y mediciones por sensor.
- Cliente de consulta: un segundo programa se adjunta a la memoria compartida existente y muestra las estadísticas sin frenar al colector.
- Terminación normal: al procesar todas las mediciones esperadas, el colector termina y libera todos los recursos IPC.

2. Arquitectura general

El sistema se compone de **tres ejecutables independientes** que se comunican exclusivamente a través de mecanismos IPC POSIX (nunca comparten memoria de proceso ni variables globales de C, salvo la memoria explícitamente mapeada con `mmap`):



Cada **sensor** es un proceso creado por el colector mediante `fork()` + `execv()`. Genera una cantidad fija de mediciones pseudoaleatorias y las envía, una por una, a la cola de mensajes POSIX con nombre `/mq_mediciones_ap3`, que ya fue creada por el colector antes del `fork`.

El **colector** es el proceso principal. Crea los tres recursos IPC (cola, memoria compartida y semáforo), lanza los procesos sensor, y levanta un *pool* de worker **threads** (con `pthread_create`) que hacen *fan-out*: todos comparten el mismo descriptor de la cola y compiten por consumir mensajes, algo que POSIX permite de forma segura porque `mq_receive()` es atómico a nivel de kernel. Cada mensaje procesado actualiza la memoria compartida dentro de una sección crítica protegida por un semáforo binario.

El **consulta** es un segundo proceso, completamente separado del colector (no comparte threads ni el mismo espacio de direcciones), que se adjunta por nombre a la memoria compartida y al semáforo **ya existentes** (sin `O_CREAT`) y muestra una foto de las estadísticas actuales. Esto es exactamente lo que demuestra que la memoria compartida es IPC real entre procesos, como pide el punto 6 del enunciado.

---

3. Decisiones de diseño y su justificación

A continuación se listan las decisiones de diseño más relevantes, cada una justificada con la *regla de oro* correspondiente de los apuntes de clase de Laboratorio III (Módulos 1 a 8).

Decisión	Justificación / regla de oro aplicada
Cola de mensajes y semáforo con la **API POSIX con nombre** (`mq_open`, `sem_open`) en vez de System V.	Módulo 4/5: “En esta materia se adopta la API moderna de POSIX (POSIX.4) en lugar de System V”. Además, POSIX identifica los recursos por nombre legible (`/mq_mediciones_ap3`), lo que evita la complejidad de `ftok`.
`mq_maxmsg = 10` al crear la cola.	Es el límite por defecto de `/proc/sys/fs/mqueue/msg_max` en Linux sin privilegios de root; usar un valor mayor rompe la entrega en máquinas sin configuración especial. El backpressure natural de `mq_send` (bloquea si la cola está llena) reemplaza cualquier necesidad de un buffer más grande.
Sensores creados con `fork()` + `execv()` (procesos reales), no simplemente threads.	Es un requisito explícito del enunciado (“el colector debe crear una cantidad configurable de **procesos** sensores”) y del Módulo 3 (gestión de procesos, `argc/argv`). Fork+exec es el patrón clásico de Unix para lanzar un programa distinto desde otro.
Sincronización de la memoria compartida con un **semáforo binario nombrado** (`sem_open`) en vez de `pthread_mutex_t`.	Los workers son threads del mismo proceso, pero **consulta** es OTRO proceso que también necesita coordinarse con la shm para leer una foto consistente. Un `pthread_mutex_t` normal no es válido entre procesos distintos. El Módulo 5 enseña exactamente esta combinación: “la memoria compartida no incluye sincronización propia; requiere asociarse obligatoriamente con semáforos”.
`sem_post()` se escribe **inmediatamente después** de cada `sem_wait()`, rellenando el código adentro.	Regla de oro del Módulo 5 (“buena práctica: escribir el sem_post inmediatamente después del sem_wait y luego rellenar el código interno”), para evitar dejar el mutex tomado ante un `return` anticipado.
Terminación de los workers mediante **“píldoras de veneno”** (mensaje de control con `sensor_id = -1`) en vez de sondeo activo o timeouts.	Aplica el mismo principio que la regla de oro del Módulo 2 sobre el anti-patrón `while(!feof(fp))`: nunca se decide el fin de la lectura “adivinando” con un chequeo externo (acá sería un `if (cola_vacía)`); se controla explícitamente con una señal de datos real dentro del mismo canal.
Chequeo obligatorio de todo valor de retorno que pueda fallar (`fork`, `mq_open`, `shm_open`, `sem_open`, `malloc`, `pthread_create`).	Regla transversal de los Módulos 2, 4 y 5: “cada vez que llamamos a fopen hay que chequear si retornó NULL”, generalizada a todo el resto de las funciones que pueden fallar (con su propio valor de error: `-1`, `NULL` o `SEM_FAILED` según corresponda).

Decisión	Justificación / regla de oro aplicada
Limpieza preventiva de recursos IPC "huérfanos" (`mq_unlink`/`shm_unlink`/`sem_unlink`) al **inicio** del colector, antes de crear los propios con `O_EXCL`.	Generaliza la regla de oro del Módulo 5 sobre semáforos huérfanos: "si se vuelve a ejecutar el programa con un valor inicial diferente, sem_open ignorará el nuevo valor... comportamientos anómalos difíciles de depurar". Se usa `O_CREAT|O_EXCL` para **garantizar** que somos nosotros quienes fijamos el valor inicial y el tamaño.
Liberación **total** de recursos al finalizar: `munmap`, `shm_unlink`, `mq_close+mq_unlink`, `sem_close+sem_unlink`.	Requisito explícito (punto 7 del enunciado) y extensión de la regla de `fclose` del Módulo 2 ("liberación del descriptor") a todos los recursos IPC usados.
`srand()` se llama **una única vez** por proceso, al inicio de `main()`, combinando `time(NULL)` con `getpid()`.	Regla de oro del Módulo 3: "srand() debe invocarse una sola vez al inicio de la ejecución… ejecutarla dentro de bucles repite los valores". Se agregó `getpid()` porque hay **varios procesos** sensor arrancando en el mismo segundo, y sin diferenciarlos podrían generar la misma secuencia.
Validación estricta de `argv`: chequeo de `argc`, mensajes de uso por `stderr` usando `argv[0]`, y salida con `EXIT_FAILURE`.	Regla de oro del Módulo 3 ("patrón de validación y seguridad"): nunca se accede a `argv[i]` sin haber verificado antes `argc`, y como `atoi`/`atol` devuelven 0 ante una entrada inválida, todo valor numérico se valida después de convertir.
C89 estricto: declaraciones al inicio de cada bloque, sólo comentarios `/* */`, `int main(int argc, char *argv[])` explícito, sin `//` ni *arrays* de tamaño variable.	Cumple las "Reglas Clave de C89" del Módulo 1 y se verificó compilando con `-std=c89 -pedantic -Wall -Werror` (build 100% limpio, ver Sección 6).
`-D_XOPEN_SOURCE=700 -D_DEFAULT_SOURCE` en el `Makefile`, y `-lpthread` ubicado **al final** del comando de enlazado.	Es exactamente la combinación de flags que muestran los apuntes del Módulo 5 para poder usar `usleep()` y las APIs POSIX.4 bajo `-std=c89` estricto, y para que el enlazador resuelva los símbolos de semáforos correctamente.

Nota sobre la sección crítica de `consulta`: el cliente de consulta toma el mismo semáforo que los workers, pero únicamente para copiar el struct completo a una variable local (una asignación de struct, instantánea); toda la impresión en pantalla se hace **después** de soltar el semáforo. Así se cumple el punto 6 del enunciado ("muestra las estadísticas actuales sin frenar al colector"): la sección crítica dura microsegundos, nunca lo que tarda un `printf`.

---

4. Guía paso a paso: qué hace cada archivo y cada función

4.1 comun.h — contrato compartido entre los tres programas

Es el único archivo que incluyen los tres ejecutables. Sigue al pie de la letra la regla de oro del Módulo 3 sobre bibliotecas propias: **en un `.h` sólo van declaraciones**, nunca definiciones, y tiene su *include guard* (`#ifndef COMUN_H`). Define:

- `MAX_SENSORES`, `DEFAULT_NUM_WORKERS`, `MAX_WORKERS`: límites del sistema (necesarios porque C89 no permite arreglos de tamaño variable en la shm).
- `MQ_NOMBRE`, `SHM_NOMBRE`, `SEM_NOMBRE`: los tres nombres POSIX (empiezan con `/`), definidos una única vez para que no haya desincronización de nombres entre archivos.
- `ID_PILDORA_VENENO`: valor centinela para las píldoras de veneno.
- `medicion_msg_t`: el mensaje que viaja por la cola (sensor_id, valor, timestamp).

- `estadistica_sensor_t` y `estadisticas_globales_t`: los campos que pide el enunciado (cantidad, suma, mínimo, máximo, último valor, por sensor y global).
- `memoria_compartida_t`: el layout exacto que se mapea con `mmap()` tanto en el colector como en consulta.
- Prototipos de `fallar()` y `valor_aleatorio()`.

4.2 comun.c — biblioteca propia de funciones utilitarias

Implementación de las dos funciones declaradas en el header (Módulo 3: separación interfaz/implementación):

- `fallar(contexto)`: centraliza el patrón obligatorio `perror()` + `exit(EXIT_FAILURE)` que exige la materia ante cualquier syscall que devuelva error.
- `valor_aleatorio(min, max)`: genera un `double` pseudoaleatorio en `[min, max)`, usado por cada sensor para simular una medición física.

4.3 sensor.c — proceso productor

Flujo de `main()` (recibe `id_sensor` y `cantidad_mediciones` por `argv`):

- Valida `argc` y convierte/valida los dos parámetros.
- `srand(time(NULL) ^ getpid())`: semilla única por proceso.
- `mq_open(MQ_NOMBRE, O_WRONLY)`: se conecta a la cola YA creada por el colector (sin `O_CREAT`: el sensor nunca crea recursos IPC, sólo los usa).
- Bucle `for` de `cantidad` iteraciones: arma un `medicion_msg_t`, lo envía con `mq_send()` y espera un retardo aleatorio (`usleep`, hasta 40 ms) que simula el muestreo real de un sensor físico y le da tiempo a `consulta` para mostrar datos en vivo.
- `mq_close()` y termina con `return 0` (el colector detecta esto con `waitpid`/`WIFEXITED`).

4.4 colector.c — proceso principal (el corazón del sistema)

`main()` sigue, en orden, estos doce pasos (todos comentados en el código fuente con el mismo numerado):

#	Paso en main()
1	Valida argc/argv (cant_sensores, mediciones_por_sensor y, opcional, cant_workers).
2	limpiar_recursos_previos(): unlink defensivo de mq/shm/sem por si quedó algo huérfano.
3	Crea la cola con mq_open(O_CREAT|O_EXCL), fijando mq_maxmsg y mq_msgsize.
4	Crea la shm con shm_open(O_CREAT|O_EXCL) + ftruncate + mmap, y la inicializa en cero.
5	Crea el semáforo binario con sem_open(O_CREAT|O_EXCL, ..., 1).
6	fork() + execv("./sensor", ...) por cada sensor configurado; guarda cada PID.
7	pthread_create() de cada worker, pasándole la cola, la shm y el mutex.
8	waitpid() de cada sensor (confirma que TODOS terminaron de enviar).
9	Envía una píldora de veneno por cada worker.
10	pthread_join() de cada worker (ya consumieron su píldora y terminaron).
11	Marca procesamiento_completo=1 dentro de la sección crítica e imprime el resumen final.
12	Libera TODOS los recursos: munmap, shm_unlink, mq_close+mq_unlink, sem_close+sem_unlink.

Funciones auxiliares (todas `static`, privadas de este archivo):

- `inicializar_memoria_compartida()`: pone en cero toda la shm y carga los valores iniciales (num_sensores_activos, total_esperadas, etc.).
- `actualizar_estadisticas()`: aplica UNA medición a las estadísticas del sensor correspondiente y a las globales (min/máx/suma/cantidad/último valor). Se llama siempre dentro de la sección crítica.
- `rutina_worker()`: cuerpo de cada worker thread — bucle infinito de `mq_receive()`; si el mensaje es la píldora, corta el bucle; si es una medición real, toma el mutex, llama a `actualizar_estadisticas()` y libera el mutex.
- `imprimir_resumen()`: recorre la shm (ya con el mutex tomado por el llamador) e imprime el reporte final por sensor y global.

4.5 consulta.c — cliente de solo lectura

- `shm_open(SHM_NOMBRE, O_RDONLY, 0)`: se adjunta a la shm EXISTENTE (sin O_CREAT). Si falla, informa por qué (probablemente el colector no está corriendo) y termina.
- `mmap(..., PROT_READ, MAP_SHARED, ...)`: mapeo de solo lectura (a nivel de hardware: cualquier intento de escritura causaría un segmentation fault, refuerzo adicional de que este proceso nunca modifica los datos del colector).
- `sem_open(SEM_NOMBRE, 0)`: se adjunta al semáforo existente (sin O_CREAT ni value, que de todos modos se ignorarían).
- Bucle principal: toma el mutex, copia el struct completo a una variable local (`snapshot = *shm;`), suelta el mutex, y recién ahí llama a `mostrar_estadisticas()`. Si se pasó un intervalo por argv, repite en modo *watch* hasta Ctrl+C.
- `mostrar_estadisticas()`: imprime, por cada sensor activo, cantidad/promedio/mínimo/máximo/último valor, y después la fila GLOBAL agregada.

4.6 Makefile

Define `CFLAGS = -std=c89 -D_XOPEN_SOURCE=700 -D_DEFAULT_SOURCE -pedantic -Wall -Werror -pthread -g` y `LDLIBS = -lrt -lpthread` (al final del comando de enlazado, como indican los apuntes). Compila cada `.c` a su propio `.o` (compilación incremental, Módulo 3) y después enlaza cada ejecutable con `comun.o`. Incluye también `make clean` y `make limpiar_ipc` (borrado manual de recursos IPC huérfanos, ver Módulo 5).

---

5. Checklist de cumplimiento del enunciado

#	Requisito	Cómo se cumple	Dónde
1	Colector crea cantidad configurable de procesos sensores	fork() + execv("./sensor", ...) en un bucle de num_sensores iteraciones	colector.c, paso 6
2	Envío por cola de mensajes	Cola POSIX con nombre /mq_mediciones_ap3, mq_send desde cada sensor	sensor.c / colector.c
3	Procesamiento concurrente (fan-out) con mutex/semáforo	Pool de N pthreads, todos comparten el mismo mqd_t; semáforo binario protege la shm	colector.c, rutina_worker()
4	Estadísticas en memoria compartida (cant., suma/prom., min, max, último, por sensor y globales)	memoria_compartida_t mapeada con mmap(); actualizar_estadisticas() calcula los 5 campos	comun.h, colector.c

#	Requisito	Cómo se cumple	Dónde
5	Configuración por argv: cant. sensores y mediciones por sensor	`./colector <cant_sensores> <mediciones_por_sensor> [cant_workers]`	colector.c, main()
6	Cliente de consulta adjunto a la shm, sin frenar al colector	consulta.c: shm_open/sem_open sin O_CREAT, sección crítica de solo copiar el struct	consulta.c
7	Terminación normal liberando todos los recursos IPC	Tras el último pthread_join: munmap+shm_unlink, mq_close+mq_unlink, sem_close+sem_unlink	colector.c, paso 12

---

6. Pruebas y verificación realizadas

Se verificó el sistema de forma incremental antes de dar por terminada la entrega:

Prueba	Resultado
Compilación con `-std=c89 -pedantic -Wall -Werror`	0 warnings, 0 errores, en los tres ejecutables.
Corrida chica (`./colector 3 5`)	15/15 mediciones procesadas, estadísticas por sensor correctas, recursos liberados.
Corrida larga (`./colector 5 200 3`) + `./consulta` en paralelo	consulta mostró una foto “EN CURSO” con datos parciales coherentes (267/1000) mientras el colector seguía corriendo, sin bloquearlo.
`./consulta` sin colector corriendo	Falla de forma controlada con mensaje explicativo (shm_open: No such file or directory), exit code 1.
Validación de argv (0 sensores, texto en vez de número, límites excedidos)	Los cinco casos de error se rechazan con mensaje por stderr y exit(EXIT_FAILURE), sin crashear.
Prueba de máxima escala (`./colector 32 50 8`)	1600/1600 mediciones procesadas correctamente, los 32 sensores presentes en el resumen.
Inspección manual de `/dev/shm` y `/dev/mqueue` durante y después de una corrida	Los archivos `shm_estadisticas_ap3`, `sem.sem_estadisticas_ap3` y la cola existen mientras corre el colector, y desaparecen apenas termina.
`valgrind --tool=helgrind` sobre una corrida completa	0 errores de condición de carrera detectados sobre el acceso concurrente a la memoria compartida.

---

7. Uso de Inteligencia Artificial

Tal como pide explícitamente la consigna, se detalla a continuación el uso de IA en la producción de esta entrega.

7.1 ¿Qué información se usó para alimentar a la IA?

- El PDF de la consigna original (*0360 APU Trabajo Integrador Parte 1 M2 262Q*), con el enunciado completo, el diagrama del sistema y los siete requisitos puntuales.
- El apunte de clase completo de Laboratorio III (Módulos 1 a 5: Linux/GCC/C89, Archivos, Procesos, IPC & POSIX, Semáforos y Productor-Consumidor), en formato Markdown, incluyendo todas sus “reglas de oro” y tablas comparativas.

- Prompts propios describiendo el pedido (“crear el programa para completar el trabajo integrador, verificando los apuntes de clase y las reglas de oro del profesor, y armar una guía paso a paso”).

7.2 ¿Para qué se la utilizó?

Se usó un asistente de IA (Claude, de Anthropic, a través de Claude Code/Cowork) para: (a) diseñar la arquitectura completa del sistema a partir del diagrama y los siete requisitos del enunciado; (b) escribir el código fuente en C89 estricto de los tres ejecutables y la biblioteca común; (c) escribir el `Makefile` con los flags de compilación exigidos por la materia; (d) compilar de forma iterativa hasta lograr cero *warnings* bajo `-Wall -Werror -pedantic`; (e) ejecutar pruebas funcionales, de estrés y de condiciones de carrera (`valgrind --tool=helgrind`); y (f) redactar este mismo informe.

7.3 Decisiones y revisión personal

Las decisiones de diseño que aparecen en la Sección 3 (semáforo nombrado en vez de mutex de pthreads, píldoras de veneno en vez de *polling*, límite de `mq_maxmsg=10`, limpieza preventiva de recursos huérfanos, etc.) fueron seleccionadas explícitamente para alinearse con las técnicas enseñadas en los apuntes de la materia, en lugar de alternativas más genéricas o “primero que aparece en internet” que la IA podría haber propuesto por defecto (por ejemplo, semaphore.h de C11, colas de System V, o `pthread_mutex_t` compartido con `PTHREAD_PROCESS_SHARED`, que también hubiera funcionado pero no es lo que enseña el curso).

Nota para la defensa oral (Parte 2): este código fue producido con asistencia de IA en una única sesión de trabajo, a partir de los dos documentos fuente mencionados arriba. Antes de la entrega final y la defensa oral, se recomienda releer cada archivo, correr las pruebas descriptas en la Sección 6 con las propias manos, y estar preparada para explicar cualquier línea del código (en particular la sincronización con semáforos y el mecanismo de píldoras de veneno), ya que la Parte 2 se defiende como examen parcial oral sobre el sistema completo.

---

8. Cómo compilar y ejecutar

Instrucciones rápidas (el detalle completo, con ejemplos de salida, está en `README.md`):

```
$ make                      # compila colector, sensor y consulta
$ ./colector 5 500 4        # 5 sensores, 500 mediciones c/u, 4 workers

# en otra terminal, mientras el colector sigue corriendo:
$ ./consulta                # una foto de las estadisticas actuales
$ ./consulta 1              # modo watch: se refresca cada 1 segundo

$ make clean                # borra binarios y objetos
$ make limpiar_ipc          # borra a mano recursos IPC huerfanos, si hiciera falta
```

`colector` y `sensor` deben estar en la misma carpeta (el colector ejecuta `./sensor` de forma relativa).